Table des matières

Labo 01 — Analyser la RAW reçue 1
Avant de commencer 2
Étape 1 — Compter la livraison contre ce qu'elle annonce 2
Étape 2 — Voir que tout est du texte 3
Étape 3 — Premier défaut : le même objet, livré plusieurs fois 4
Étape 4 — Deuxième défaut : des codes que personne n'a décodés 4
Étape 5 — Troisième défaut : des textes qui ne sont pas du texte 5
Étape 6 — Créer le dépôt `02_ae` 5
Dépannage 6

Labo 01 — Analyser la RAW reçue

Labo · Cours 1/9 — Formation Analytics Engineering

But : compter la couche `RAW` de Boréal contre le document qui l'annonce, voir de vos yeux les trois défauts qu'elle garde par contrat, puis créer `02_ae`, le dépôt de vos neuf cours. Le pourquoi de chaque étape est dans `Theorie_01_AE.md`, section 7 et annexe A.

Deux outils :

- l'Anaconda Prompt, environnement `formations` actif : Git et les commandes du système ;
- l'interface web de DuckDB (`duckdb -ui`), un notebook de requêtes dans le navigateur : le SQL.

Liste de contrôle :

- ☐ Cours 00 fait, mise en situation lue
- ☐ Livraison comptée : 22 tables, 513 857 lignes, 1 572 fichiers, 0 écart avec `LIVRAISON.md`
- ☐ Tout en texte : 250 colonnes `VARCHAR`
- ☐ Les trois défauts vus : 26 538 lignes pour 25 483 commandes, `KUKLA` à `01`, `AEDAT` à `00000000`
- ☐ Un maximum faux constaté : 9 996,26 en texte au lieu de 293 478,95

© 2026 iAlexMG. Tous droits réservés.

☐ Notebook fermé ; à vous : dépôt `02_ae` créé, `.gitignore` validé, `git status` propre

---

Avant de commencer

La couche `RAW` vient du Cours 00, `Cours_00_AE_Boreal.md` : si `Projet_Boreal_intermediaire` manque, le faire d'abord. Anaconda Prompt, environnement `formations` actif, placée dans `Projet_Boreal_intermediaire` :

```
dir /b plateforme\entrepot
REM Attendu : boreal.duckdb.
```

L'Anaconda Prompt reste dans `Projet_Boreal_intermediaire` pour tout le labo : aucun `cd` à taper ensuite.

Lire `02_Boreal_intermediaire/Mise_en_situation_Boreal_02_AE.md`.

Étape 1 — Compter la livraison contre ce qu'elle annonce

Théorie : `Theorie_01_AE.md`, annexe A.1.

1.1 Lancer l'interface, depuis l'Anaconda Prompt.

```
duckdb -ui
```

- L'interface s'ouvre dans le navigateur, à `http://localhost:4213`. Au premier lancement, DuckDB télécharge son extension `ui` : il faut Internet.
- Laisser l'Anaconda Prompt ouvert : le fermer arrête l'interface.
- Créer un notebook (`+` à côté de Notebooks) et le renommer « AE Labo 01 » (double-clic sur son titre, puis Entrée) : chaque labo aura le sien.
- Une requête par cellule (`Add Cell`). Ctrl+Entrée l'exécute. Une cellule qui contient plusieurs requêtes n'affiche que le dernier résultat.
- Les lignes `--` donnent le résultat attendu ; DuckDB les ignore.

1.2 Attacher l'entrepôt en lecture seule, dans la première cellule.

```
-- Attendu : boreal paraît sous Attached databases.
attach if not exists 'plateforme/entrepot/boreal.duckdb' as boreal (read_only);
```

Chaque requête qui suit va dans une cellule neuve et nomme ses tables en entier (`boreal.raw.erp_vbak`).

1.3 Faire l'inventaire du schéma `raw`, puis le comparer, table par table, à `01_Data_Engineering/02_Boreal_intermediaire/Solution/LIVRAISON.md`, section 2.2.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 22 lignes, de corpus_index (33) à usine_etiquettes (58), dont erp_vbak (26 538).
-- Chaque compte égale celui de LIVRAISON.md.
select table_name, estimated_size as lignes_estimees
from duckdb_tables()
where database_name = 'boreal' and schema_name = 'raw'
order by table_name;
```

1.4 Totaliser la couche.

```sql
-- Attendu : 22 tables, 513 857 lignes, le total de LIVRAISON.md.
select count(*) as tables_raw, sum(estimated_size) as lignes_raw
from duckdb_tables()
where database_name = 'boreal' and schema_name = 'raw';
```

1.5 Compter exactement les cinq tables de l'ERP. `union all` empile les cinq comptes en un seul résultat (Cours 07 de la formation SQL avancé, `Theorie_07_SQL.md`, section 6).

```sql
-- Attendu : erp_kna1 7 481, erp_mara 2 459, erp_vbak 26 538, erp_vbap 81 292, erp_zserie 32 597.
select 'erp_kna1' as table_raw, count(*) as lignes from boreal.raw.erp_kna1
union all select 'erp_mara', count(*) from boreal.raw.erp_mara
union all select 'erp_vbak', count(*) from boreal.raw.erp_vbak
union all select 'erp_vbap', count(*) from boreal.raw.erp_vbap
union all select 'erp_zserie', count(*) from boreal.raw.erp_zserie
order by 1;
```

1.6 Compter les fichiers que le chargement a lus.

```sql
-- Attendu : 1 572, le total de LIVRAISON.md.
select count(*) as fichiers_charges from boreal.pipeline.chargements;
```

Étape 2 — Voir que tout est du texte

Théorie : `Theorie_01_AE.md`, annexe A.2.

2.1 Compter les colonnes du schéma `raw` par type.

```sql
-- Attendu : 3 lignes. VARCHAR 250, TIMESTAMP WITH TIME ZONE 22, JSON 2.
select data_type, count(*) as colonnes
from information_schema.columns
where table_catalog = 'boreal' and table_schema = 'raw'
group by 1
order by 2 desc;
```

2.2 Lire les trois colonnes de lignage d'une table.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 3 lignes, chacune avec le chemin de son fichier (sources/erp/extraits/…/VBAK.csv),
-- son lot (un mois) et l'heure de votre chargement.
select _fichier, _lot, _charge_le
from boreal.raw.erp_vbak
limit 3;
```

Étape 3 — Premier défaut : le même objet, livré plusieurs fois

Théorie : `Theorie_01_AE.md`, annexe A.3.

3.1 Compter les lignes et les commandes distinctes des en-têtes de commande.

```sql
-- Attendu : 26 538 lignes, 25 483 commandes, 1 055 lignes en trop.
select
    count(*) as lignes,
    count(distinct vbeln) as commandes,
    count(*) - count(distinct vbeln) as lignes_en_trop
from boreal.raw.erp_vbak;
```

3.2 Compter les lignes des lots de mars 2023.

```sql
-- Attendu : 609 au lot 2023-03, 609 au lot 2023-03_bis.
select _lot, count(*) as lignes
from boreal.raw.erp_vbak
where _lot like '2023-03%'
group by 1
order by 1;
```

3.3 Suivre une commande livrée trois fois.

```sql
-- Attendu : 3 lignes. Lot 2023-02 : aedat 00000000, netwr 6079.02.
-- Lots 2023-03 et 2023-03_bis : aedat 20230302, netwr 5556.72.
select vbeln, _lot, erdat, aedat, netwr
from boreal.raw.erp_vbak
where vbeln = '0010006931'
order by _lot;
```

Étape 4 — Deuxième défaut : des codes que personne n’a décodés

Théorie : `Theorie_01_AE.md`, annexe A.4.

4.1 Compter les codes de palier des concessionnaires.

© 2026 iAlexMG. Tous droits réservés.

```
-- Attendu : 01 1 127, 02 2 566, 03 3 788.
select kukla as code_palier, count(*) as lignes
from boreal.raw.erp_kna1
group by 1
order by 1;
```

4.2 Ouvrir `Data_Domain/_univers/boreal/DICTIONNAIRE.md`, à la section de `KUKLA`, et lire ce que valent les trois codes.

Attendu : `01` Or, `02` Argent, `03` Bronze.

Étape 5 — Troisième défaut : des textes qui ne sont pas du texte

Théorie : `Theorie_01_AE.md`, annexe A.5.

5.1 Compter les dates de dernière modification vides.

```
-- Attendu : false 2 308, true 24 230.
select aedat = '00000000' as date_vide, count(*) as lignes
from boreal.raw.erp_vbak
group by 1
order by 1;
```

5.2 Convertir la date, sans son format, puis avec.

```
-- Attendu : 0 date par le cast direct, 2 308 avec le format.
select
    count(try_cast(aedat as date)) as cast_direct,
    count(try_strptime(aedat, '%Y%m%d')) as avec_le_format
from boreal.raw.erp_vbak;
```

5.3 Chercher la plus grosse commande, en texte, puis en nombre.

```
-- Attendu : 9996.26 en texte, 293478.95 en nombre.
select
    max(netwr) as max_texte,
    max(netwr::decimal(18, 2)) as max_nombre
from boreal.raw.erp_vbak;
```

Étape 6 — Créer le dépôt `02_ae`

Théorie : `Theorie_01_AE.md`, annexe A.6.

6.1 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans l'Anaconda Prompt.

© 2026 iAlexMG. Tous droits réservés.

```
.quit
```

Attendu : l'invite de l'Anaconda Prompt revient, toujours dans `Projet_Boreal_intermediaire`. Le notebook « AE Labo 01 » reste enregistré.

6.2 À vous : créer le dépôt `02_ae` et son `.gitignore`, puis valider le fichier, sans quitter `Projet_Boreal_intermediaire`. C'est le geste du Cours 03 de La Brûlerie (sous-étapes 1.1 et 7.1), sans son code. Le `.gitignore` écarte deux familles :

- les artefacts de dbt, qui se refont par une commande : `target/`, `dbt_packages/`, `logs/`, `.user.yml` ;
- la connexion, qui ne va jamais dans Git : `profiles.yml`, `.env`.

Attendu : `git -C 02_ae status` répond `nothing to commit, working tree clean`, et `git -C 02_ae log --oneline` montre un seul commit. L'avertissement `LF will be replaced by CRLF` est sans conséquence. Le corrigé est `02_Analytics_Engineering/02_Boreal_intermediaire/Solution/02_ae/.gitignore`, qui porte en plus une exception ajoutée au Cours 08.

Dépannage

Symptôme	Correctif
`IO Error: Cannot open file "…\plateforme\entrepot\boreal.duckdb": Le chemin d'accès spécifié est introuvable.` en 1.2	`duckdb -ui` n'a pas été lancé dans `Projet_Boreal_intermediaire` : `.quit`, se placer dans `Projet_Boreal_intermediaire`, relancer ; si le fichier manque, faire le Cours 00
`IO Error: Cannot open file … utilisé par un autre processus` en 1.2	un autre programme tient l'entrepôt (une ligne de commande DuckDB, un script) : le fermer, puis relancer la cellule
`Binder Error: Catalog "boreal" does not exist!`	l'entrepôt n'est pas attaché (interface relancée) : relancer la cellule de 1.2
La cellule n'affiche qu'un seul résultat	elle contient plusieurs requêtes : une requête par cellule
La page ne répond plus	l'Anaconda Prompt a été fermé : relancer `duckdb -ui` depuis `Projet_Boreal_intermediaire`, puis la cellule de 1.2
L'extension `ui` ne s'installe pas	pas d'Internet au premier lancement : se connecter, puis relancer
Un compte de l'étape 1 ne tombe pas juste	le Cours 00 ne s'est pas rendu à `Conforme` : le refaire
`git status` montre des fichiers inattendus	`git init` a été lancé ailleurs que dans `Projet_Boreal_intermediaire` : supprimer le `.git` créé par erreur, recommencer

© 2026 iAlexMG. Tous droits réservés.